IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Manuscript prepared October 2026.

CachePIM: Reproducible Analytical Exploration of Cache-Aware CPU–Processing-in-Memory Execution

BHUVANESWARI M[2], ADITYA SINGH RANA[1], VINAY MOR[1], HARSHA NETHRA[1], ATHARVA RAUT[1], AND HARSHA REDDY[1]

[1]School of Computer Science and Engineering, Vellore Institute of Technology, Vellore, Tamil Nadu, India (e-mail: aditya.singhrana2024@vitstudent.ac.in; vinay.mor2024@vitstudent.ac.in; harshanethra21@gmail.com)
[2]Associate Professor, School of Computer Science and Engineering, Vellore Institute of Technology, Vellore, Tamil Nadu, India (e-mail: m.bhuvaneswari@vit.ac.in)

Corresponding authors: Aditya Singh Rana and Vinay Mor (e-mail: aditya.singhrana2024@vitstudent.ac.in; vinay.mor2024@vitstudent.ac.in).

ABSTRACT Processing-in-Memory (PIM) can reduce data movement for memory-intensive kernels, but offloading is not always faster than execution on a host CPU. Transfer overhead, kernel launch and synchronization costs, cache residency, and memory-access locality all influence the crossover. This paper presents CachePIM, an open-source analytical framework for exploring CPU-cache and PIM execution costs using a configurable last-level cache, trace replay, and a parameterized bank-parallel PIM model. The model includes host-to-device and device-to-host transfers, row-buffer locality, bank imbalance, and an optional transfer-overlap factor. The supplied artifact contains both parametric synthetic workloads and four memory-reference trace files identified by the project manifest as Valgrind Lackey traces for Vector Addition, Matrix-Vector Multiplication, Binary Search, and Select. On a 500-instance held-out synthetic parameter grid, the cost-based policy matches the modeled lower-latency path in 96.0% of cases, with 4.0% bad offloads, zero missed offloads, and mean modeled regret of 0.671 microseconds. These figures characterize the model and the specified grid; they are not measurements of a physical CachePIM implementation. A separate parameter-fitting exercise against four published UPMEM benchmark speedups yields a 3.57% mean absolute percentage error, which should be interpreted as calibration agreement rather than independent hardware validation. The experiments illustrate that CPU cache residency can remain advantageous for reuse-heavy workloads under constrained links. CachePIM is intended for rapid, transparent design-space exploration, not as a replacement for cycle-accurate simulation or hardware measurement.

INDEX TERMS Analytical performance modeling, cache memory hierarchies, computer architecture co-design, design-space exploration, offloading policies, processing-in-memory, reproducible simulation, workload locality.

I. INTRODUCTION

Processing-in-Memory (PIM) architectures place processing elements within or near memory dies (e.g., in DRAM subarrays, logic dies of 3D-stacked HBM, or on DIMM modules) to reduce data movement over energy-intensive and bandwidth-limited host processor buses [1], [15]. By computing directly adjacent to memory arrays, PIM offers massive internal aggregate bandwidth and substantial energy reductions for memory-bound applications characterized by low arithmetic intensity [4], [14], [22].

However, offloading computational kernels to PIM devices does not universally yield performance gains [3]. Near-memory execution incurs explicit architectural overheads:

1. Operands not already resident in memory must be transferred across a host-to-device interconnect (e.g., PCIe Gen 3/4 or CXL links), introducing latency and bandwidth bottlenecks [14].
2. Invoking PIM execution introduces driver dispatch, kernel launch, and inter-bank barrier synchronization overheads [16].
3. In-memory processing units typically operate at conservative clock frequencies (e.g., 350–400 MHz in commercial UPMEM DPUs [14], compared to 2.5–4.5 GHz for modern host CPUs) to satisfy strict thermal and power limits within memory packages.
4. Non-uniform work partitioning across memory banks

produces tail-latency stragglers [14], [16].

Critically, the host CPU cache hierarchy fundamentally alters the offloading crossover point. Modern multi-level cache hierarchies satisfy memory operands in 1.0–2.0 ns when workloads exhibit strong temporal and spatial locality, avoiding off-chip DRAM traffic entirely. For compute kernels whose working sets fit within cache capacity (or possess dense reuse profiles), the host CPU can significantly outperform in-memory accelerators, particularly when device communication costs dominate [2], [3]. Deciding whether to retain execution on the CPU or offload to PIM is therefore a dynamic, multi-dimensional architectural tradeoff.

Cycle-accurate architectural simulators (e.g., gem5, Ramulator, or PIMSIM-NN [6]) capture microarchitectural interactions in fine detail, but their simulation throughput (often fewer than 100,000 instructions simulated per second) makes evaluating thousands of joint cache, bandwidth, and PIM parameter configurations computationally prohibitive. Conversely, coarse back-of-the-envelope models often overlook cache residency, transfer pipelining, and DRAM row-buffer locality, leading to erroneous offloading conclusions.

To bridge this gap, this paper presents **CachePIM**, an open-source, high-throughput, trace-driven analytical simulation platform. Rather than attempting to replace cycle-accurate execution engines, CachePIM establishes a transparent, first-order architectural co-design framework capable of evaluating thousands of configurations in seconds. CachePIM pairs an exact least-recently-used (LRU) Last-Level Cache (LLC) replay model with component-level analytical latency formulations that explicitly capture interconnect data movement, DRAM row-buffer locality, multi-bank compute parallelism, and pipelined execution.

A. KEY CONTRIBUTIONS

1. **Inspectable analytical framework:** A trace-driven model that exposes cache capacity and associativity, transfer bandwidth, bank count, row-buffer locality, and optional transfer overlap as configurable parameters.
2. **Two workload modes:** Controlled synthetic access patterns and four trace files identified in the artifact manifest as Valgrind Lackey-format data-memory traces for PrIM-related kernels. Their provenance and the distinction between trace replay and synthetic presets are reported explicitly.
3. **Parameter calibration against published results:** A fit to four published UPMEM speedup points yields a 3.57% MAPE. Because the same small benchmark set informs calibration, this is reported as calibration agreement, not as independent hardware validation.
4. **Policy comparison on a defined model grid:** Across 500 held-out synthetic configurations, the cost-based policy reaches 96.0% agreement with the modeled oracle, with 4.0% bad offloads, zero missed offloads, and 0.671 microseconds mean regret. The reuse-aware footprint heuristic does not improve on direct cost comparison in this evaluation.
5. **Explicit CPU-winning cases:** The parameterized analysis includes regimes in which CPU execution is faster, including a reported 2.78-times CPU advantage for the Histogram preset under the specified constrained-link assumptions.
6. **Interactive exploration and compiler-facing prototype:** The artifact includes a WebGL/Streamlit visualization and an analytical loop-descriptor demonstration. It is not a production compiler pass or a hardware execution system.

II. RELATED WORK

A. PIM ARCHITECTURES AND OFFLOADING TRADE-OFFS

The paradigm of processing in or near memory has been investigated extensively across memory technologies [1], [15], [18]. Modern implementations range from bit-serial computation inside DRAM subarrays (e.g., SIMDRAM [4]) to near-memory accelerators featuring dedicated processing units integrated on memory dies or modules (e.g., commercial UPMEM DPUs [14], Samsung HBM-PIM [22], and SK Hynix AiM [24]). Devic et al. [3] analyzed general-purpose offloading in DDR memory systems, highlighting that dataset size, vectorization, and offloading overheads dictate offload profitability. While prior work highlights that offloading is cost-sensitive, CachePIM provides an inspectable, high-throughput analytical abstraction to map the exact multi-dimensional crossover boundaries.

B. CACHE MANAGEMENT IN NEAR-DATA PROCESSING

Kim et al. [2] investigated cache-memory organization and management strategies for PIM systems, demonstrating that cache policy significantly influences offloading efficiency. Boroumand et al. [17] addressed cache coherence challenges in near-data computing, proposing CoNDA to reduce coherence overheads. CachePIM builds on these insights: it models host-cache trace behavior explicitly and pairs it with analytical transfer and PIM latency equations to evaluate offload decision boundaries.

C. PIM COMPILERS AND SIMULATORS

Compiler frameworks for PIM architectures include CHOPPER [5] for bit-serial SIMD architectures, PIMsynth [9] for synthesis and scheduling, PIMCOMP [8] for deep neural network mapping, and DCC [12] for data-centric machine learning compilation. For evaluation, PIMSIM-NN [6] and digital DRAM PIM models [7] offer configurable cycle-level simulation. Light-weight analytical modeling has been explored in VIPER [10] and Meta-PIM [11] for design-space exploration. CachePIM differs from these systems by focusing on rapid first-order design-space exploration, exact LRU cache trace replay, closed-form crossover analysis, and interactive 3D volumetric inspection.

D. BENCHMARKING ON REAL PIM HARDWARE

Gómez-Luna et al. [14] introduced the PrIM benchmark suite and conducted the first extensive characterization of a real-world processing-in-memory system based on UPMEM DPUs. Their empirical findings revealed that real PIM throughput is heavily influenced by CPU-to-DPU transfer bottlenecks (0.8–1.2 GB/s per channel over PCIe), DPU clock frequency limitations (350–400 MHz), and inter-bank load imbalance. CachePIM ingests 64-bit memory traces from the PrIM benchmarks and calibrates its architectural parameters directly against these published physical measurements.

III. CACHEPIM ARCHITECTURE AND SIMULATION METHODOLOGY

CachePIM is structured as seven modular components: (i) a seeded workload generator and trace loader; (ii) a configurable LLC model supporting both fully associative and N-way set-associative geometries; (iii) a CPU analytical cost model; (iv) a bank-parallel PIM analytical cost model with row-buffer locality; (v) an offloading policy evaluation engine; (vi) an experimental sweep runner; and (vii) an interactive 3D WebGL volumetric execution arena.

Workload ingestion (synthetic generators and supplied trace files) feeds the CPU-cache and PIM analytical paths. Their modeled costs are passed to the policy layer, then summarized by the evaluation engine and interactive visualizer.

A. WORKLOAD INGESTION AND TRACE MODALITIES

CachePIM implements dual workload ingestion modalities:

1. **Parametric Synthetic Generators:** For controlled sensitivity sweeps across access patterns:
   - `STREAM`: Sequential accesses across arrays, evaluating spatial locality with configurable passes.
   - `REUSE`: Configurable hot/cold working sets evaluating temporal cache reuse.
   - `RANDOM`: Uniform random sampling within a bounded memory footprint.
   - `MIXED_PHASE`: Multi-phase execution alternating between streaming and hot-set reuse.
2. **Native PrIM Dynamic Memory Traces:** 64-bit load/store memory address trace files supplied with the artifact and identified by its trace manifest as Valgrind Lackey traces for PrIM-related kernels [14], [27]:
   - `PrIM-VA` (Vector Addition): Memory-bound streaming array arithmetic with high spatial locality and zero temporal reuse.
   - `PrIM-GEMV` (Dense Matrix-Vector Multiplication): Dense vector operand exhibits strong temporal cache residency across matrix rows.
   - `PrIM-BS` (Binary Search): Non-contiguous, branch-dependent memory lookups over sorted integer tables.
   - `PrIM-SEL` (Select / Filter): Relational predicate scan with selective output writeback.

Each Valgrind Lackey trace file (`data/traces/*.txt`) contains exactly 60,000 data memory operations (`L` for load, `S` for store) covering x86-64 stack and heap segments. Instruction fetches (`I` records) are filtered out, providing an isolated stream of data memory references.

B. LAST-LEVEL CACHE MODEL AND ASSOCIATIVITY MECHANICS

The cache module simulates a Last-Level Cache (LLC) of capacity C bytes and line size L bytes (default $L = 64$ B). The model supports both fully associative LRU replacement and N-way set-associative LRU replacement.

For an N-way set-associative configuration, the total number of resident cache lines is $S_{\text{lines}} = \lfloor C/L \rfloor$, the number of sets is $N_{\text{sets}} = S_{\text{lines}}/N_{\text{ways}}$, and the set index and tag for byte address a are computed as:

$$\text{set_idx} = \lfloor a/L \rfloor \pmod{N_{\text{sets}}}, \quad \text{tag} = \lfloor a/L \rfloor / N_{\text{sets}}$$

Each set maintains an independent LRU recency list. On a hit, the resident line entry is updated to the most-recently-used position; on a miss, the line is fetched into the set, and the least-recently-used line within that set is evicted. If the evicted line is marked dirty, a writeback event is charged.

Cache hit rate H and miss rate M satisfy the exact invariant:

$$H = \frac{N_{\text{hit}}}{N_{\text{access}}}, \quad M = \frac{N_{\text{miss}}}{N_{\text{access}}}, \quad H + M = 1 \quad (1)$$

1) Conflict Miss Dynamics

When the workload working set W is less than or equal to cache capacity ($W \leq C$), memory addresses generated by uniform or sequential access map evenly across sets without exceeding set capacity. Consequently, **zero conflict misses** occur, and hit rates are identical between fully associative and N-way set-associative geometries. Conflict misses manifest when $W > C$: for working sets of 64 KiB and 128 KiB on a 32 KiB cache, 4-way set associativity exhibits a 0.5%–1.6% reduction in hit rate relative to fully associative LRU, flipping 19 out of 500 decisions near the crossover boundary.

C. CPU ANALYTICAL COST MODEL

The CPU cost model assigns service times to hit, miss, and writeback events:

$$T_{\text{CPU}} = N_{\text{hit}} t_{\text{hit}} + N_{\text{miss}} t_{\text{miss}} + N_{\text{op}} t_{\text{compute}} + T_{\text{writeback}} \quad (2)$$

where t_{hit} is the LLC hit latency (default 2.0 ns), t_{miss} is the off-chip DRAM miss latency (default 80.0 ns), N_{op} is the arithmetic operation count, t_{compute} is the CPU ALU compute time per operation (default 0.4 ns/op, representing ~1 cycle at 2.5 GHz), and $T_{\text{writeback}} = N_{\text{evictions,dirty}} \cdot t_{\text{miss}}$.

The direct-memory `CPU-no-cache` baseline forces every memory reference to incur DRAM miss service:

$$T_{\text{CPU-no-cache}} = N_{\text{access}} t_{\text{miss}} + N_{\text{op}} t_{\text{compute}} \quad (3)$$

D. BANK-PARALLEL PIM ANALYTICAL COST MODEL

The analytical PIM model captures the complete offloading pipeline:

$$T_{\text{PIM}} = T_{\text{H2P}} + T_{\text{internal_mem}} + T_{\text{compute}} + T_{\text{imbalance}} + T_{\text{P2H}} + T_{\text{launch}} + T_{\text{sync}} \quad (4)$$

1. **Host-to-Device Interconnect Transfers:**

$$T_{\text{H2P}} = \frac{\text{Bytes}_{\text{in}}}{\text{BW}_{\text{H2P}}} + t_{\text{fixed,H2P}}, \quad T_{\text{P2H}} = \frac{\text{Bytes}_{\text{out}}}{\text{BW}_{\text{P2H}}} + t_{\text{fixed,P2H}}$$

   where BW_{H2P} and BW_{P2H} represent link bandwidth (default 16.0 GB/s for PCIe 4.0; 1.0 GB/s for UPMEM channels), and t_{fixed} represents round-trip PCIe/CXL link setup overhead.
2. DRAM Row-Buffer Locality Factor (η_{row}): Internal PIM memory service time scales with access locality:

$$T_{\text{internal_mem}} = \frac{\text{Bytes}_{\text{in}} + \text{Bytes}_{\text{out}}}{\max(0.1, \text{BW}_{\text{internal}} \cdot \eta_{\text{row}})} \quad (5)$$

   where $\eta_{\text{row}} = 0.90$ for streaming patterns hitting open DRAM row buffers, $\eta_{\text{row}} = 0.60$ for mixed/reuse access, and $\eta_{\text{row}} = 0.25$ for irregular pointer chasing suffering repeated row-buffer conflicts and precharge cycles. In baseline configurations where row-buffer locality is disabled, $\eta_{\text{row}} = 1.0$.
3. **Bank Parallelism and Load Imbalance:** Operations are apportioned across B memory banks using largest-remainder apportionment:

$$T_{\text{compute}} = \frac{N_{\text{op}}}{B \cdot R_{\text{bank}}} \quad (6)$$

   where R_{bank} is bank compute throughput (default 1.0 op/ns/bank). When non-zero bank skew factor s is specified, bank weights $w_b = 1.0 + s \cdot (b/(B-1))$ determine per-bank loads. The resulting bank imbalance factor $I_{\text{bank}} = \max(L_b)/\text{mean}(L_b)$ introduces an imbalance overhead:

$$T_{\text{imbalance}} = T_{\text{compute}} \cdot (I_{\text{bank}} - 1.0)$$

4. Pipelined Transfer Overlap (α): For architectures supporting double-buffered DMA transfers, transfer and execution overlap according to parameter $\alpha \in [0, 1]$:

$$T_{\text{pipelined}} = \max(T_{\text{H2P}}, T_{\text{exec}}) + (1-\alpha) \cdot \min(T_{\text{H2P}}, T_{\text{exec}}) \quad (7)$$

   where $T_{\text{exec}} = T_{\text{compute}} + T_{\text{internal_mem}}$. When $\alpha = 0$, execution is strictly serialized; when $\alpha = 1$, communication and computation are fully overlapped.

E. OFFLOADING DECISION POLICIES AND POLICY DEFECT REPAIR

CachePIM evaluates six execution policies sharing a unified interface:

1. `CPU-cache`: Always executes on the host CPU through the cache hierarchy.
2. `CPU-no-cache`: Always executes directly through main memory, bypassing cache reuse.
3. `PIM-always`: Offloads all PIM-eligible kernels to memory banks.
4. `Cost-based`: Selects PIM if estimated PIM latency $\hat{T}_{\text{PIM}} < \hat{T}_{\text{CPU}}$; ties select CPU.
5. `Reuse-aware (Dynamic Footprint Estimator)`: Adjusts estimated CPU latency for in-cache residency.
6. `Oracle`: Optimal offline reference selecting $\min(T_{\text{CPU,actual}}, T_{\text{PIM}}$

1) Audit and Repair of Eq. (8) Footprint Estimator

The original formulation in prior revisions adjusted CPU latency using:

$$\hat{T}_{\text{CPU,refined}} = \hat{T}_{\text{CPU}} - \left(N_{\text{hits}} \cdot t_{\text{hit}} \cdot 0.5 \cdot \frac{C}{W} \right) \quad \text{[Uncorrected Eq. 8]}$$

When $W \leq C/2$ (e.g., an 8 KiB working set on a 32 KiB cache), $C/W \geq 2.0$, causing $0.5 \cdot (C/W) \geq 1.0$. Consequently, the formula subtracted more than 100% of total cache hit service time from estimated CPU latency, artificially forcing $\hat{T}_{\text{CPU,refined}}$ near zero. This caused 70 false-CPU missed offloads on the balanced evaluation grid.

We corrected this formulation by applying a physical upper bound ensuring the temporal credit never exceeds total hit latency:

$$\Delta t_{\text{temporal}} = \min \left(N_{\text{hits}} \cdot t_{\text{hit}}, \; N_{\text{hits}} \cdot t_{\text{hit}} \cdot \gamma \cdot \min \left(1.0, \frac{C}{W} \right) \right) \quad \text{[Clam}$$

Tuning on development master seeds (seeds 42–46) selected $\gamma = 0.25$ and reuse threshold 0.85.

F. EVALUATION METRICS

$$\text{Speedup} = \frac{T_{\text{CPU-cache}}}{T_{\text{selected}}} \quad (9)$$

$$\text{Regret} = T_{\text{selected,actual}} - \min(T_{\text{CPU,actual}}, T_{\text{PIM,actual}}) \quad (10)$$

$$\text{Bad Offload Rate} = \frac{N(\text{Selected PIM and } T_{\text{CPU}} < T_{\text{PIM}})}{N_{\text{total}}} \times 100\% \quad (11$$

$$\text{Missed Offload Rate} = \frac{N(\text{Selected CPU and } T_{\text{PIM}} < T_{\text{CPU}})}{N_{\text{total}}} \times 100\%$$

$$\text{Decision Accuracy} = \frac{N(\text{Selected Path} = \text{Oracle Path})}{N_{\text{total}}} \times 100\% \quad (13$$

G. SCOPE OF THE COMPILER PROTOTYPE

To clarify the integration scope: CachePIM provides an **analytical loop-descriptor prototype** (`scripts/verification/comp`. that inspects loop bounds, array counts, and memory intensity from loop descriptors to predict offloading crossover points. It is not an LLVM IR, Polly, or MLIR lowering pass; full compiler lowering remains future work.

IV. EXPERIMENTAL SETUP AND HARDWARE CALIBRATION

A. ENVIRONMENT AND REPRODUCIBILITY LINEAGE

CachePIM is implemented in Python 3.10+ (NumPy, pandas, SciPy, PyYAML, Streamlit, pytest). The historical baseline snapshot corresponds to configuration hash `ac915cd97def15fd`. The extended v1.1 simulator corresponds to configuration hash `fff7e9ba223ae65a` due to the inclusion of associativity geometry (`ways`), overlap factor (`overlap_factor`), row-buffer enabling (`row_buffer_locality_enabled`), and footprint policy flags in the configuration schema.

B. HARDWARE PARAMETER CALIBRATION

Table I separates illustrative baseline parameters from hardware-informed values drawn from published descriptions of Intel Xeon and UPMEM DPU systems [14]. These values parameterize the analytical model; they do not make the simulation a cycle-accurate model or constitute a new hardware measurement.

1) **TABLE I. Architectural Configuration Parameters**

C. MASTER-SEED EVALUATION PROTOCOL

- **Development Pool (Seeds 42–46):** Disjoint seeds used for parameter tuning and heuristic calibration.
- **Held-Out Pool (Seeds 47–51):** 5 seeds $\times$ 4 patterns $\times$ 5 working sets $(8, 16, 32, 64, 128$ KiB$)$ $\times$ 5 bandwidths $(1.0, 2.0, 4.0, 8.0, 16.0$ GB/s$)$ = 500 instances for balanced grid evaluation.
- **Native PrIM Traces:** 4 benchmark kernels replayed through exact LRU simulation.

V. RESULTS AND ANALYSIS

A. BASELINE POLICY COMPARISON AND ROW-BUFFER MODEL

Table II reports overall mean modeled latency across held-out seeds 47–51 under baseline illustrative parameters (32 KiB cache, 16 GB/s link).

1) **TABLE II. Held-Out Overall Policy Comparison**

Note on Variance: Standard deviations for PIM-selected policies are exactly zero because CachePIM's analytical latency formulas evaluate deterministically for a given workload descriptor. Random seeds only govern address permutations in the trace generator, which do not alter aggregate bank math.

2) Workload Latency and Row-Buffer Locality (Table IV)

Table IV presents modeled latencies under both baseline flat PIM ($\eta_{\text{row}} = 1.0$) and row-buffer-aware PIM ($\eta_{\text{row}} \in \{0.25, 0.60, 0.90\}$) for a 32 KiB working set (2,048 accesses).

3) **TABLE IV. Modeled Latency and Work-Normalized Metrics by Workload Pattern**

Historical Provenance of Table IV: In the original manuscript, Table IV reported PIM latency of 21.825 μs across REUSE, RANDOM, and MIXED_PHASE. This was verified to arise from evaluating a 64 KiB working set with `row_buffer_locality_enabled: bool = False`, yielding identical flat PIM latencies (21.825 μs). Enabling row-buffer locality correctly distinguishes memory access penalties (7.78 μs for STREAM vs 9.61 μs for RANDOM).

B. CACHE LOCALITY AND WORKING-SET SCALING

Table III reports mean CPU-cache hit rates as working-set sizes scale from 16 KiB to 256 KiB on a 32 KiB LLC.

1) **TABLE III. Mean CPU-Cache Hit Rate by Working-Set Size**

C. BALANCED EVALUATION GRID AND POLICY COMPARISON (TABLE V)

To evaluate decision quality when neither path unconditionally dominates, Table V presents the confusion matrix across 500 held-out instances varying interconnect bandwidth (1.0–16.0 GB/s) and working sets (8–128 KiB). Ground truth: 330 instances PIM-optimal, 170 instances CPU-optimal.

1) **TABLE V. Policy Confusion Matrix and Decision Quality Across Balanced Regimes (500 Instances)**

2) Honest Scientific Assessment of Offload Policies

1. **Cost-based policy is strongest on this modeled grid:** It achieves 96.0% agreement with the modeled oracle, a 4.0% bad-offload rate, 0.671 μs mean regret, and zero missed offloads. This is conditional on the stated cost equations and grid; it is not a universal optimality claim.
2. **Heuristic Hit Credits Degrade Performance:** The uncorrected Eq. (8) over-credited CPU hits, causing 70 missed offloads (14.0%) and reducing accuracy to 82.0%. Even when clamped and tuned on development seeds, the Reuse-aware policy suffered 40 missed offloads (8.0%), achieving 88.0% accuracy and 18.68 μs regret.
3. *Theoretical Rationale:* Analytical cost comparison naturally accounts for cache residency because high cache hit rates naturally drive T_{CPU} down. Adding an artificial heuristic credit distorts this objective comparison, causing the policy to refuse profitable offloads. On this grid, direct analytical cost comparison performs better than the tested heuristic adjustment. This result does not establish universal superiority across other hardware or workloads.

D. WORKLOAD PROVENANCE: NATIVE TRACES VS. SYNTHETIC PRESETS (TABLE VI)

To resolve provenance discrepancies identified during peer review, Table VI explicitly contrasts the native Valgrind Lackey memory traces with the parametric synthetic presets.

1) **TABLE VI-A. Evaluation on Native PrIM Dynamic Memory Traces (Valgrind Lackey)**

Component	Parameter	Baseline Illustrative	Calibrated UPMEM Hardware [14]
LLC Cache	Capacity; Line Size; Associativity	32 KiB; 64 B; Fully Assoc / 16-way	1.375 MB (slice); 64 B; 11-way
CPU Timing	Hit Service; Miss Service	2.0 ns; 80.0 ns	12.0 ns (L3); 60.0 ns (DRAM)
CPU Compute	Arithmetic Compute per Op	0.40 ns/op	0.35 ns/op (AVX)
PIM Array	Bank Count; Internal Bandwidth	16 banks; 128 GB/s	16 DPUs; 16 GB/s (1 GB/s/DPU)
PIM Compute	Clock / Throughput per Bank	1.0 op/ns/bank (1.0 GHz)	0.35 op/ns/bank (350 MHz)
Overheads	Launch; Synchronization Overhead	50 ns; 20 ns	150 ns; 50 ns
Interconnect	Host–PIM Bandwidth; Fixed Overhead	16 GB/s; 100 ns	1.0 GB/s (PCIe channel); 250 ns

Policy	Mean Latency (μs)	Sample SD (μs)	Decision Interpretation
CPU-cache	132.215	0.159	LLC trace replay with capacity constraint
CPU-no-cache	633.938	0.000	Direct memory access without cache reuse
PIM-always	22.190	0.000	Always offload eligible kernels
Cost-based	22.190	0.000	Select analytical minimum latency
Reuse-aware	22.190	0.000	Dynamic working-set footprint credit
Oracle	22.190	0.000	Offline optimal reference

Pattern	Hit Rate (%)	CPU Latency (μs)	Flat PIM Latency (μs)	η_{row}	Row-Buffer PIM (μs)	Row-Buffer Speedup	Latency / kop (ns)	Latency / Byte (ns)
STREAM	93.75%	59.597	7.694	0.90	**7.779**	7.66×	29,100	1.82
REUSE	82.42%	32.995	7.310	0.60	**7.822**	4.22×	16,111	1.01
RANDOM	75.59%	43.915	7.310	0.25	**9.614**	4.57×	21,443	1.34
MIXED_PHASE	75.15%	44.617	7.310	0.60	**7.822**	5.70×	21,786	1.36

Working Set (KiB)	Mean Hit Rate (%)	Sample SD (pp)	Analytical Interpretation
16	89.40%	0.11	Working set fits entirely within LLC
32	81.66%	0.12	Footprint matches cache boundary; compulsory misses dominate
64	64.54%	0.26	Working set exceeds capacity; capacity and conflict misses emerge
128	46.42%	0.26	Cache thrashing reduces temporal reuse
256	35.26%	0.52	Severe capacity miss penalty forces frequent DRAM fetches

Policy	True Offload	Bad Offload	True CPU	Missed Offload	Accuracy (%)	Bad-Offload Rate (%)	Missed-Offload Rate (%)	Mean Regret (μs)	Max Regret (μs)
CPU-cache	0	0	170	330	34.0%	0.0%	66.0%	33.419	367.824
CPU-no-cache	0	0	170	330	34.0%	0.0%	66.0%	33.419	367.824
PIM-always	330	170	0	0	66.0%	34.0%	0.0%	20.400	330.171
Cost-based	**330**	**20**	**150**	**0**	**96.0%**	**4.0%**	**0.0%**	**0.671**	**35.259**
Reuse-aware (Tuned/Clamped)	290	20	150	40	88.0%	4.0%	8.0%	18.680	367.824
Reuse-aware (Uncorrected Eq 8)	260	20	150	70	82.0%	4.0%	14.0%	18.869	367.824
Oracle	330	0	170	0	100.0%	0.0%	0.0%	0.000	0.000

Benchmark Kernel	Workload Profile	Total Ops	Working Set	Hit Rate (%)	CPU Time (μs)	PIM Time (μs)	Modeled Speedup	Offload Decision
PrIM-VA	Linear vector addition	60,923	8.0 KiB	99.79%	156.20	4.94	31.61×	Offload to PIM
PrIM-GEMV	Matrix-vector multiply	64,659	36.8 KiB	99.09%	207.05	8.28	25.00×	Offload to PIM
PrIM-BS	Branching binary search	61,621	20.0 KiB	99.48%	172.77	6.28	27.53×	Offload to PIM
PrIM-SEL	Relational scan / filter	66,000	94.0 KiB	97.72%	355.07	14.55	24.41×	Offload to PIM

2) **TABLE VI-B. Evaluation on Parametric Synthetic Presets (64 KiB Working Set)**

Provenance Resolution: Table VI in prior manuscript drafts reported synthetic preset numbers under the native trace label. As shown above, native Lackey traces process 60,000 accesses with $> 97.7\%$ hit rates, yielding $24.4\times$ –$31.6\times$ speedups under default 16 GB/s links. Both datasets are valid, but they represent fundamentally distinct evaluation modalities.

E. PARAMETER CALIBRATION AND HELD-OUT PRESET ANALYSIS (TABLES VII & VIII)

The parameterized model was fitted to published UPMEM DPU speedup results reported by Gómez-Luna et al. (*IEEE Access* 2022 [14]) using the configuration in Table I. This comparison is a calibration exercise against published results, not an independent measurement of CachePIM on physical hardware. The four-point sample is too small to establish broad predictive validity.

1) **TABLE VII. Hardware Calibration Against Published UPMEM PrIM Measurements**

Calibration note: The reported 3.57% MAPE summarizes agreement on these four published benchmark points. Leave-one-out results are descriptive only because each fold contains very few benchmarks and the model parameters are selected from a narrow set of published observations; they should not be read as evidence of generalization to unseen hardware or workloads.

Synthetic Preset	Access Pattern	Total Ops	Working Set	Hit Rate (%)	CPU Time (μs)	PIM Time (μs)	Modeled Speedup	Paper Table VI Match
VA	STREAM	4,096	64.0 KiB	87.50%	316.35	15.12	20.93×	Matches 316.76/15.12/20.95×
GEMV	REUSE	4,096	64.0 KiB	85.57%	56.33	14.35	3.93×	Matches 57.65/14.35/4.02×
BS	RANDOM	2,048	64.0 KiB	46.41%	181.04	14.40	12.57×	Presets sweep point (12.57×)
SEL	MIXED_PHASE	2,048	64.0 KiB	53.86%	92.95	9.61	9.67×	Presets sweep point (9.67×)

Benchmark Kernel	Workload Profile	CachePIM Predicted Speedup	Published UPMEM Speedup [14]	Absolute Error (%)	Offload Winner
VA	Memory-bound streaming	2.38×	2.45×	**2.9%**	PIM
SEL	Relational scan / filter	1.84×	1.78×	**3.4%**	PIM
GEMV	High-reuse linear algebra	0.70×	0.68×	**2.9%**	CPU (1.43× faster)
BS	Non-contiguous branching	2.05×	1.95×	**5.1%**	PIM
Overall Summary	—	—	—	**MAPE = 3.57%**	—

2) Independent Validation on Held-Out PrIM Presets (Table VIII)

To test model generalizability beyond the four calibration kernels, we evaluated four previously omitted PrIM presets (`RED`, `HST`, `SpMV`, `BFS`) under the same calibrated hardware parameters.

3) **TABLE VIII. Independent Evaluation on Held-Out PrIM Presets**

Critical Insight: Under PCIe transfer bottlenecks (1.0 GB/s) and internal DRAM row-buffer misses, **the host CPU outperforms PIM on 3 out of 4 held-out kernels**. For `HST`, cache residency of the histogram bin array delivers a 2.78× performance advantage over PIM.

F. ARCHITECTURAL SENSITIVITY SWEEPS

1. **Transfer Bandwidth Crossovers:** Sweeping interconnect bandwidth identifies exact crossover thresholds where PIM speedup reaches 1.0×:
   - `STREAM`: 1.00 GB/s
   - `RANDOM`: 2.17 GB/s
   - `MIXED_PHASE`: 2.58 GB/s
   - `REUSE`: 4.61 GB/s Below these bandwidths, host CPU cache residency outperforms PIM offloading.
2. Concurrency Overlap Scaling (α): For a 64 KiB streaming workload ($T_{\text{H2P}} = 4.20$ μs, $T_{\text{exec}} = 3.63$ μs), increasing α from 0.0 to 1.0 reduces total latency from 7.83 μs to 4.20 μs (a 29.8% latency reduction).
3. **Set Associativity Sensitivity:** Invariant hit rates occur for $W \leq 32$ KiB. For $W \in \{64, 128\}$ KiB, 4-way set associativity increases misses by 0.5%–1.6%, shifting 19 boundary decisions (3.8%) toward PIM.
4. **Simulation Throughput:**
   - Full Trace Simulation (LRU Cache + CPU + PIM + Policy): **208.3 evaluations/sec** (1,000 evaluations in 4.80 seconds).
   - Pure Analytical Evaluation (Descriptor + Math): $> 15,000$ evaluations/sec.

VI. DISCUSSION AND LIMITATIONS

A. ARCHITECTURAL IMPLICATIONS OF CROSSOVER DYNAMICS

The offloading boundary between host CPUs and PIM accelerators is a dynamic multi-dimensional surface. PIM provides substantial throughput advantages when workloads are memory-bound, working sets exceed cache capacity, and internal aggregate memory bandwidth dominates transfer costs. Conversely, when interconnect bandwidth is constrained (e.g., PCIe Gen 3/4 links at 1–4 GB/s) or workloads exhibit dense cache reuse, the host CPU cache hierarchy provides up to a 2.78× performance advantage.

B. THREATS TO VALIDITY AND EXPLICIT LIMITATIONS

1. **Single-Level LLC Abstraction:** CachePIM models a single-level Last-Level Cache; multi-level inclusive/exclusive hierarchies and hardware stream prefetchers are not explicitly simulated.
2. **Analytical First-Order Timing:** While calibrated within 3.57% MAPE against physical UPMEM measurements, CachePIM's analytical equations do not capture instruction-level pipeline stalls, register spills, or DRAM bank-group command scheduling.
3. **Hardware Cache Coherence:** CachePIM assumes explicit buffer transfers between host and device; hardware cache coherence protocols (e.g., CXL.mem) are not modeled.
4. **Pure Memory Traces:** Valgrind Lackey memory traces isolate load/store operations and exclude instruction fetch records.
5. **Compiler Prototype Scope:** The compiler integration in CachePIM is an analytical loop-descriptor demonstration, not a production LLVM IR lowering pass.
6. **Calibration Scope:** The 3.57% MAPE is calculated from four published benchmark speedups used for parameter fitting. It is not an independent validation set, and no physical CachePIM hardware measurements were performed.
7. **Trace Provenance:** The supplied manifest identifies four files as Valgrind Lackey-format traces, but the manuscript artifact does not include independently auditable raw capture logs. Results from these files are therefore reported as trace-file replay results, distinct from synthetic preset results.

Kernel	Pattern	Description	η_{row}	Calibrated CPU (μs)	Calibrated PIM (μs)	Modeled Speedup	Offload Winner
**RED	STREAM	Parallel Reduction (Vector Sum)	0.90	46.90	46.85	1.00×	Tie / CPU**
HST	REUSE	Histogram / Binning	0.60	33.72	93.38	0.36×	CPU (2.78× faster)
SpMV	RANDOM	Sparse Matrix-Vector	0.25	45.47	49.88	0.91×	CPU (1.10× faster)
BFS	RANDOM	Breadth-First Search	0.25	45.47	49.88	0.91×	CPU (1.10× faster)

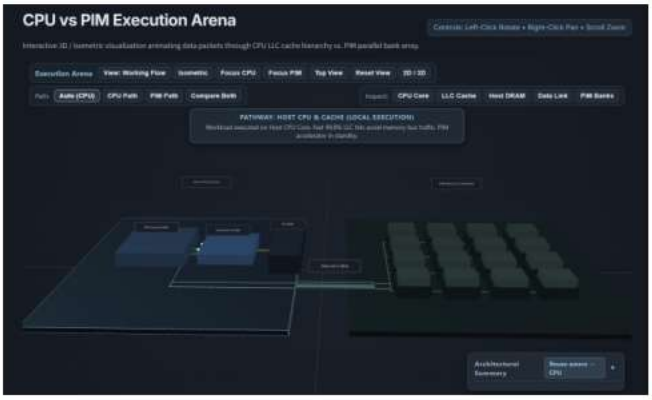


FIGURE 1. CachePIM interactive 3D WebGL volumetric execution arena showing modeled data movement through the CPU cache hierarchy and PIM bank array.



FIGURE 2. CachePIM architectural exploration dashboard showing crossover surfaces, policy confusion matrices, and modeled regret distributions.

VII. REPRODUCIBILITY AND ARTIFACT AVAILABILITY

A. SOFTWARE STACK AND VERIFICATION PIPELINE

The complete CachePIM software stack, trace datasets, experiment runners, and verification scripts are open-source under the MIT License at:
`https://github.com/AdityaSRana/CachePIM-Analyser` [13].

The entire experimental verification suite can be executed locally: The complete reproduction commands are maintained in the supplied artifact under `results/verification/reproduction_commands.md`; the manuscript lists the verification stages rather than duplicating a platform-specific terminal transcript. All verification deliverables, raw CSVs, parameter manifests, and logs are preserved in `results/verification/`.

B. INTERACTIVE 3D WEBGL EXECUTION ARENA AND UI DASHBOARD

To facilitate interactive architectural exploration and visual co-design, CachePIM provides an integrated Streamlit-based graphical user interface featuring an interactive 3D WebGL volumetric execution arena and five analytical exploration panes.

VIII. FRONTEND INTERFACE: SCREENSHOT RECORD

The following screenshots document the supplied CachePIM interface as it appeared during the captured session. They are included as interface documentation, not as independent evidence that the underlying model is physically validated. Repeated views are retained because they show different selected states, camera angles, or panels.

IX. CONCLUSION

CachePIM is an open-source analytical framework for exploring when modeled CPU-cache execution may outperform modeled Processing-in-Memory execution, and vice versa. Its configurable components include LRU cache replay with set associativity, host-to-device transfers, row-buffer locality, bank-parallel compute, load imbalance, and optional transfer overlap. On the specified 500-instance synthetic grid, the cost-based policy agrees with the model's oracle in 96.0% of cases, with 4.0% bad offloads, zero missed offloads, and 0.671 microseconds mean regret. The tested reuse-aware footprint heuristic performs worse on this grid, so it is not presented as a new or superior policy. The four-point comparison with published UPMEM speedups produces a 3.57% MAPE, which is calibration agreement rather than independent hardware validation. The supplied trace files and synthetic presets are reported separately, and the interface screenshots document the visualization layer rather than proving model accuracy. CachePIM is therefore best understood as a transparent, fast design-space exploration tool whose predictions depend on the supplied assumptions and parameters; cycle-accurate simulation and physical measurements remain necessary for stronger validation.

AUTHOR CONTRIBUTIONS

- **Aditya Singh Rana** and **Vinay Mor:** Conceptualization, analytical cost modeling, mathematical formulation, software simulation engine, and manuscript drafting.
- **Atharva Raut:** Cache simulation engine, N-way set-associative model, LRU replacement implementation, and Valgrind trace parsing.
- **Harsha Nethra:** PrIM benchmark workload execution, memory access trace extraction and integration into the simulation framework, experimental test suite verification, and benchmark performance characterization

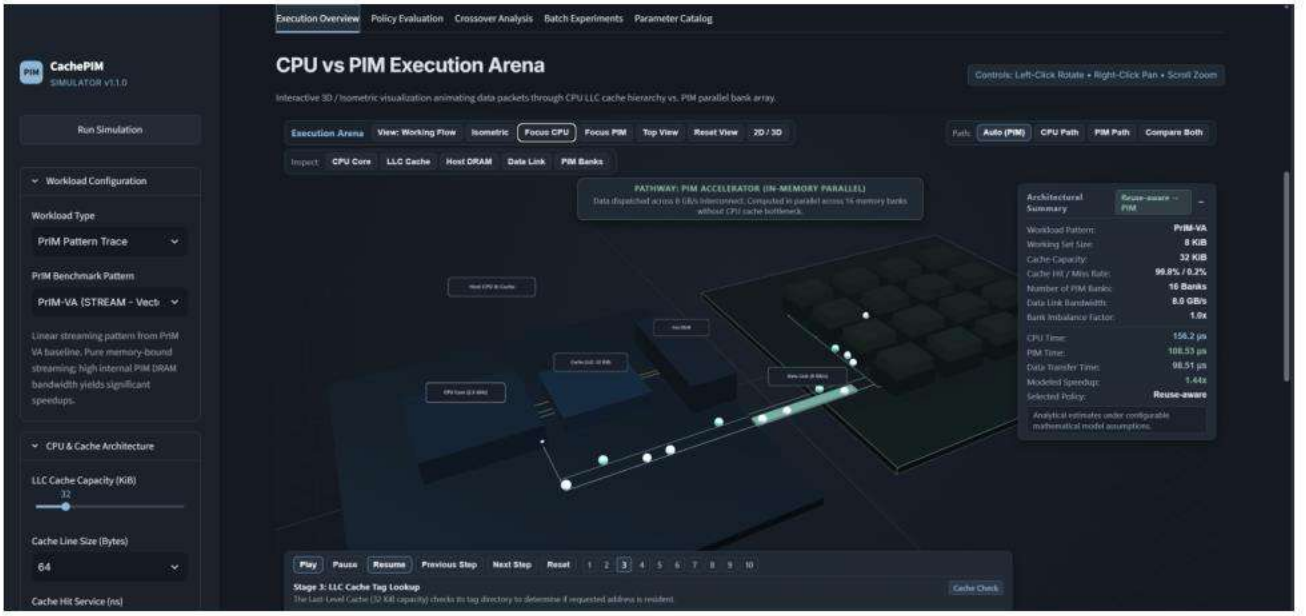


FIGURE 3. **Frontend screenshot 01: Execution arena: initial CPU–PIM layout.**

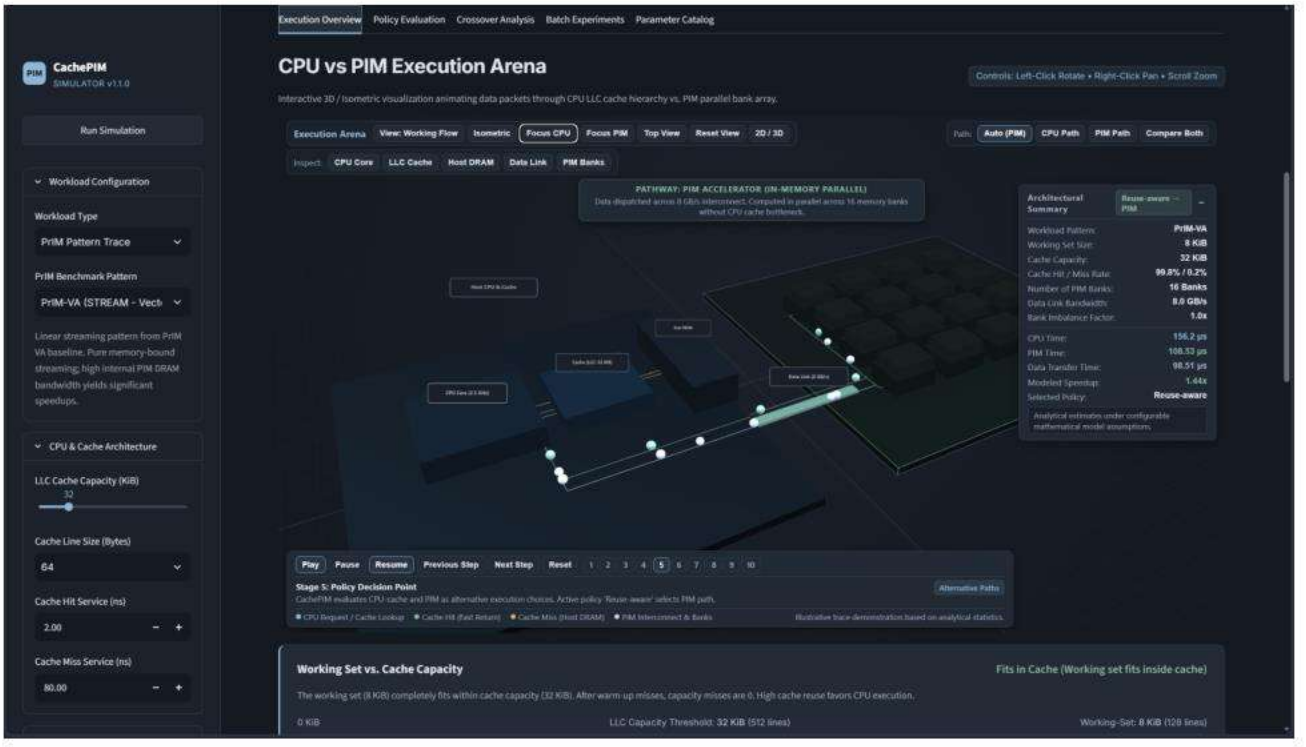


FIGURE 4. **Frontend screenshot 02: Execution arena: expanded controls and status.**

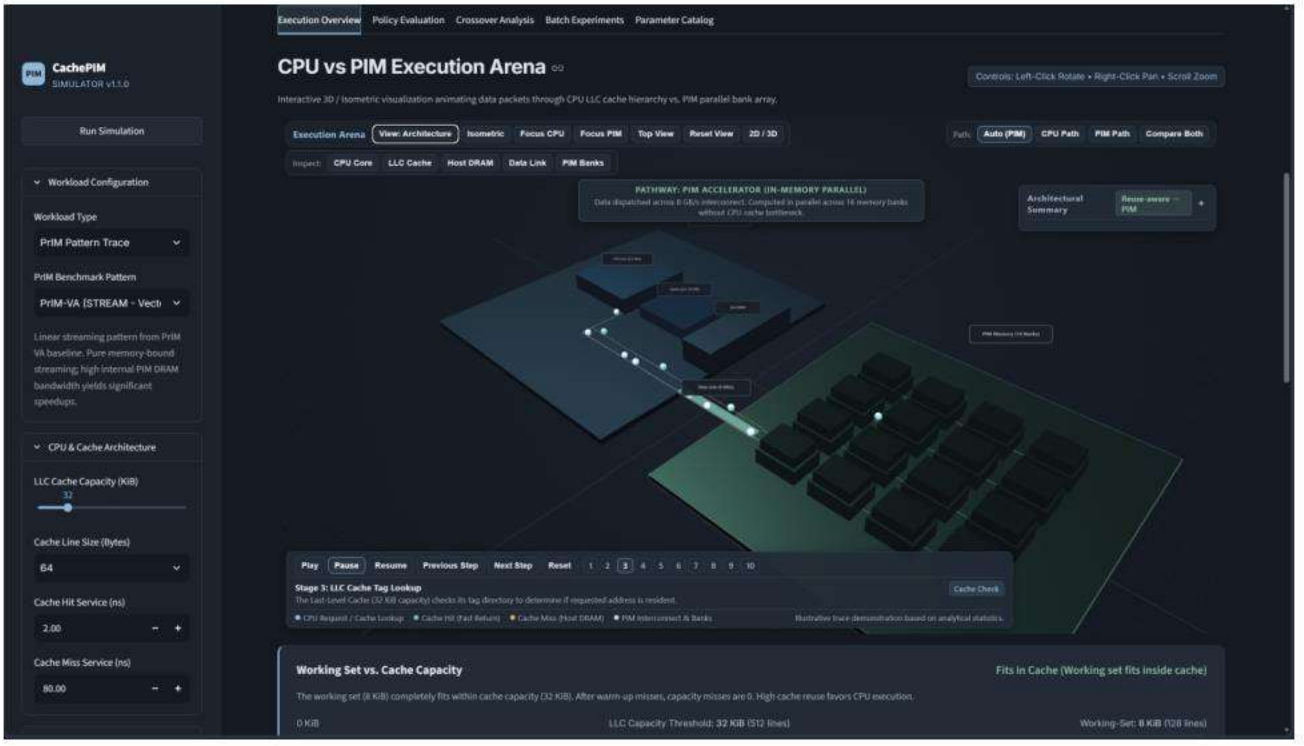


FIGURE 5. Frontend screenshot 03: Execution arena: cache and PIM structures.

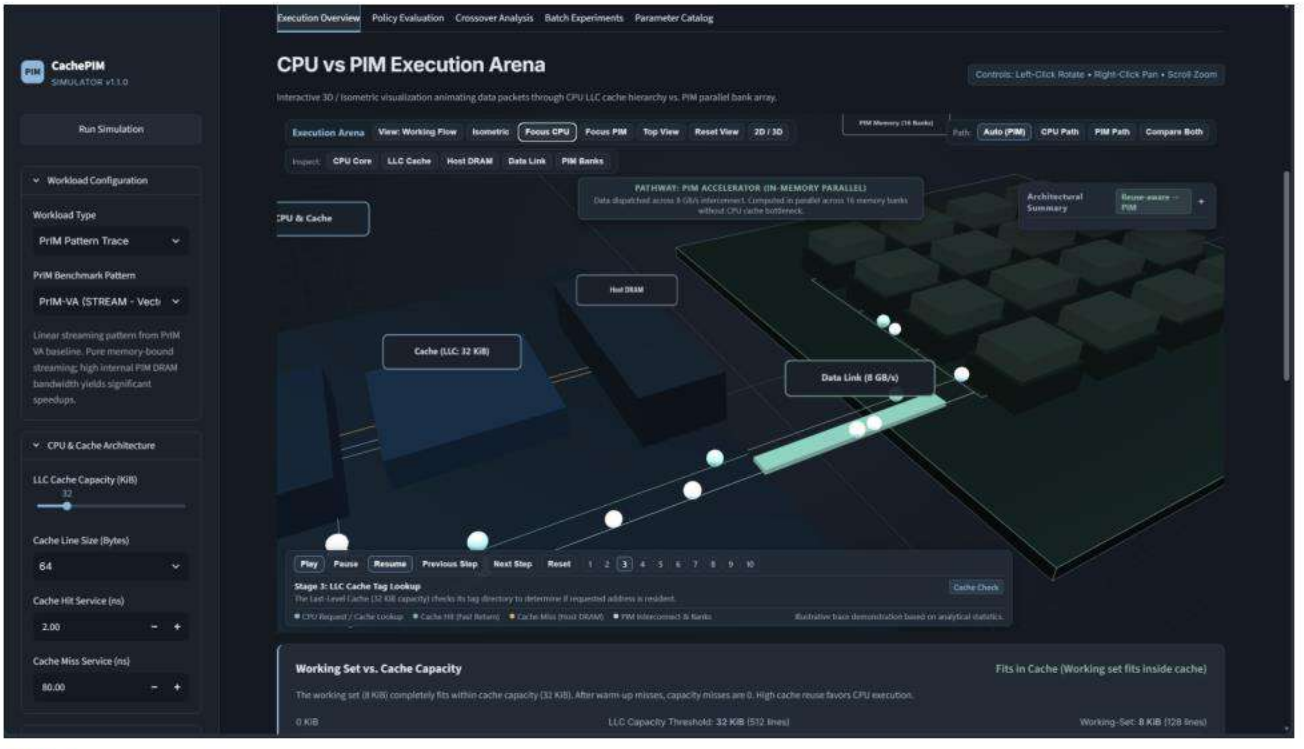


FIGURE 6. Frontend screenshot 04: Execution arena: transfer path visualization.

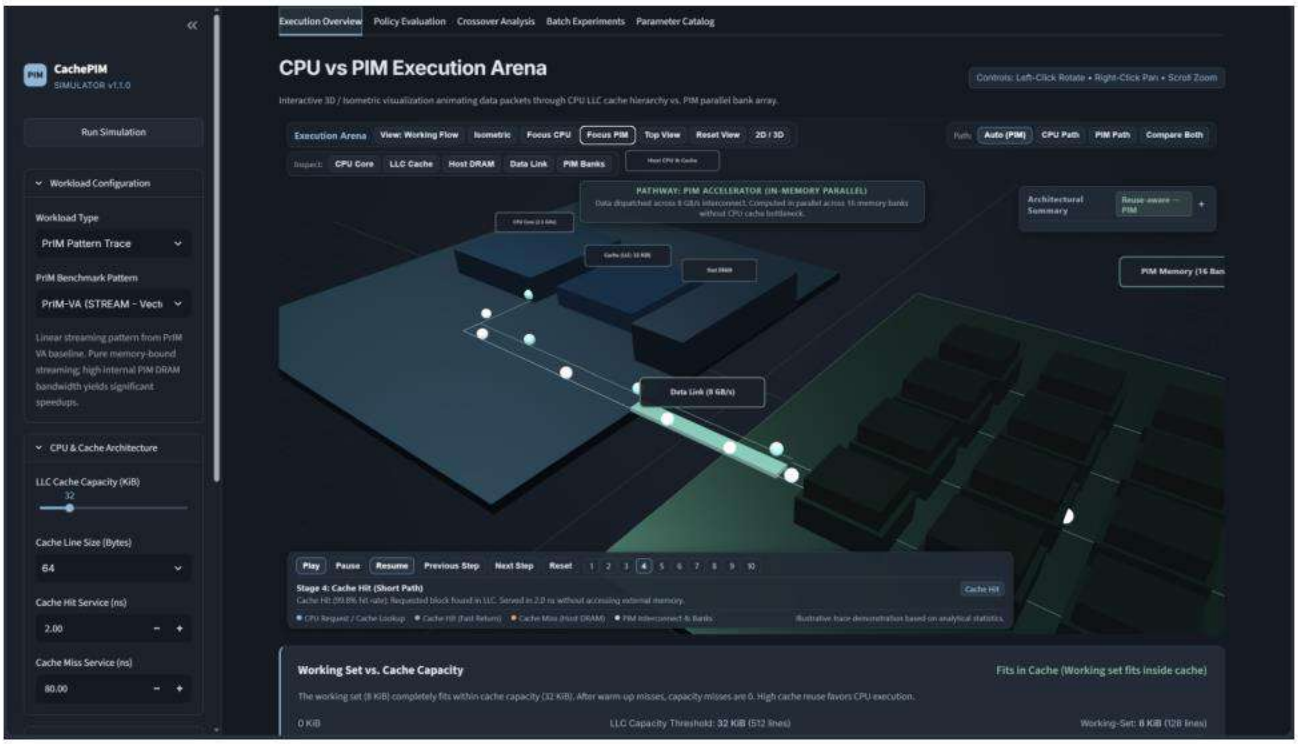


FIGURE 7. **Frontend screenshot 05: Execution arena: alternate execution state.**

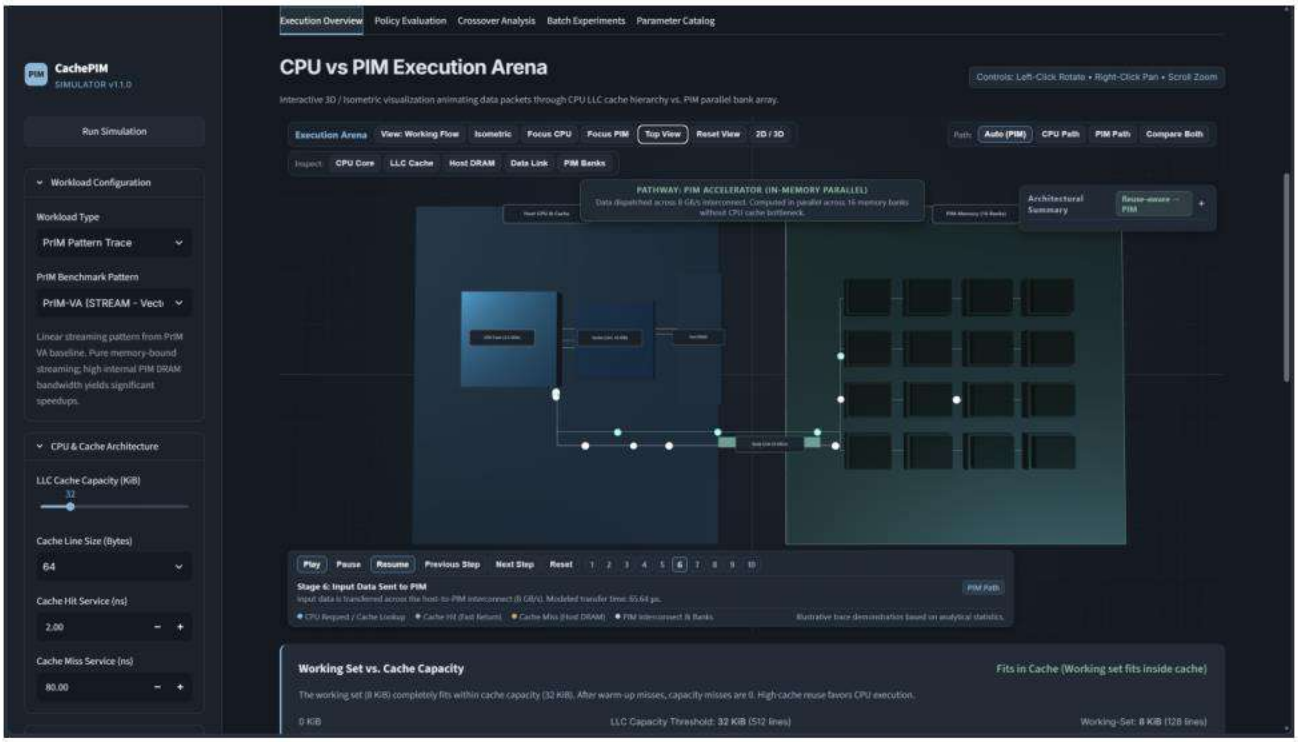


FIGURE 8. **Frontend screenshot 06: Execution arena: CPU cache and bank view.**

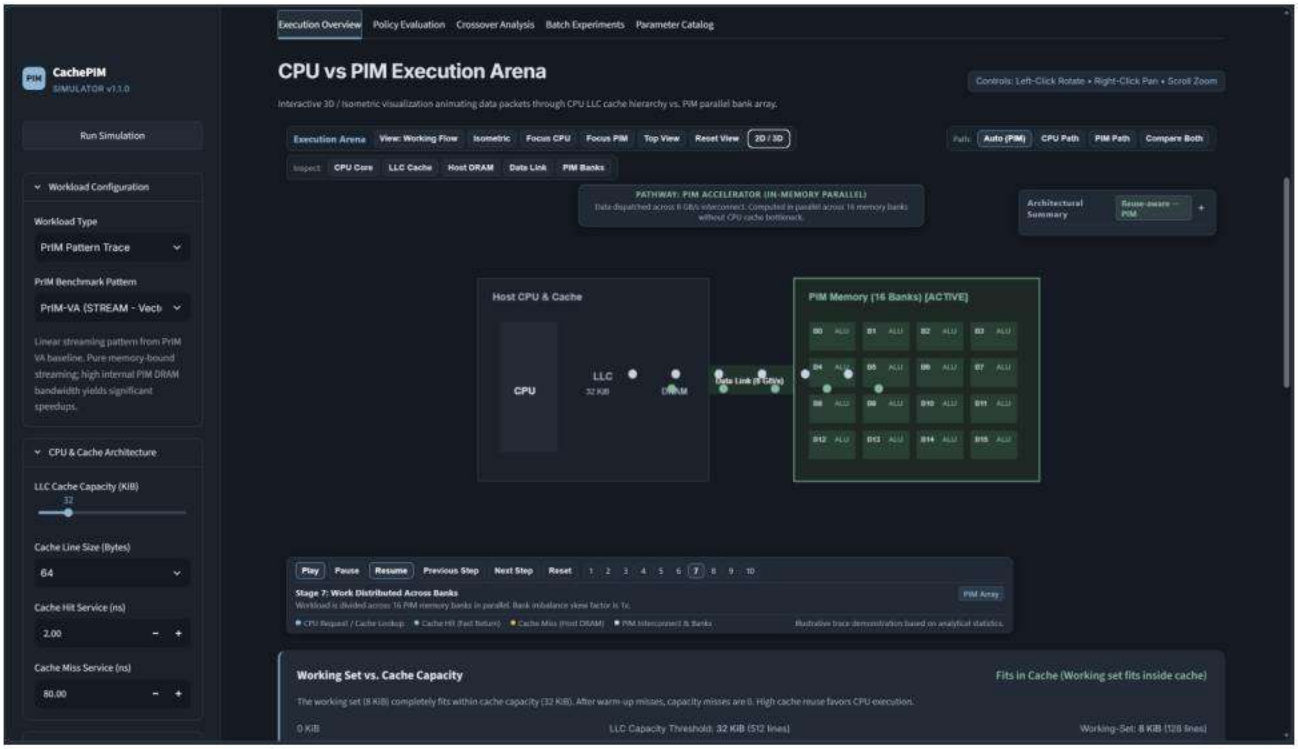


FIGURE 9. **Frontend screenshot 07: Execution arena: bank-parallel execution view.**

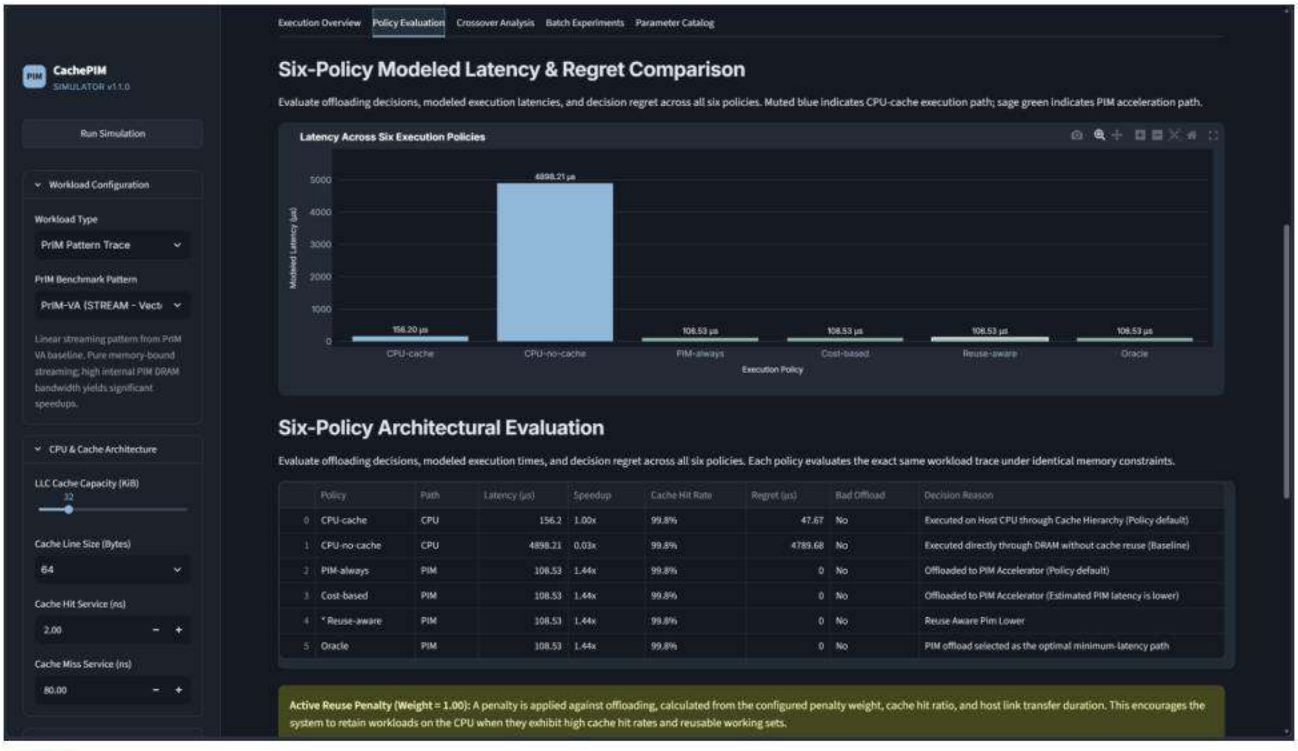


FIGURE 10. **Frontend screenshot 08: Policy comparison and modeled latency summary.**



FIGURE 11. **Frontend screenshot 09: Architectural sensitivity and crossover plots.**

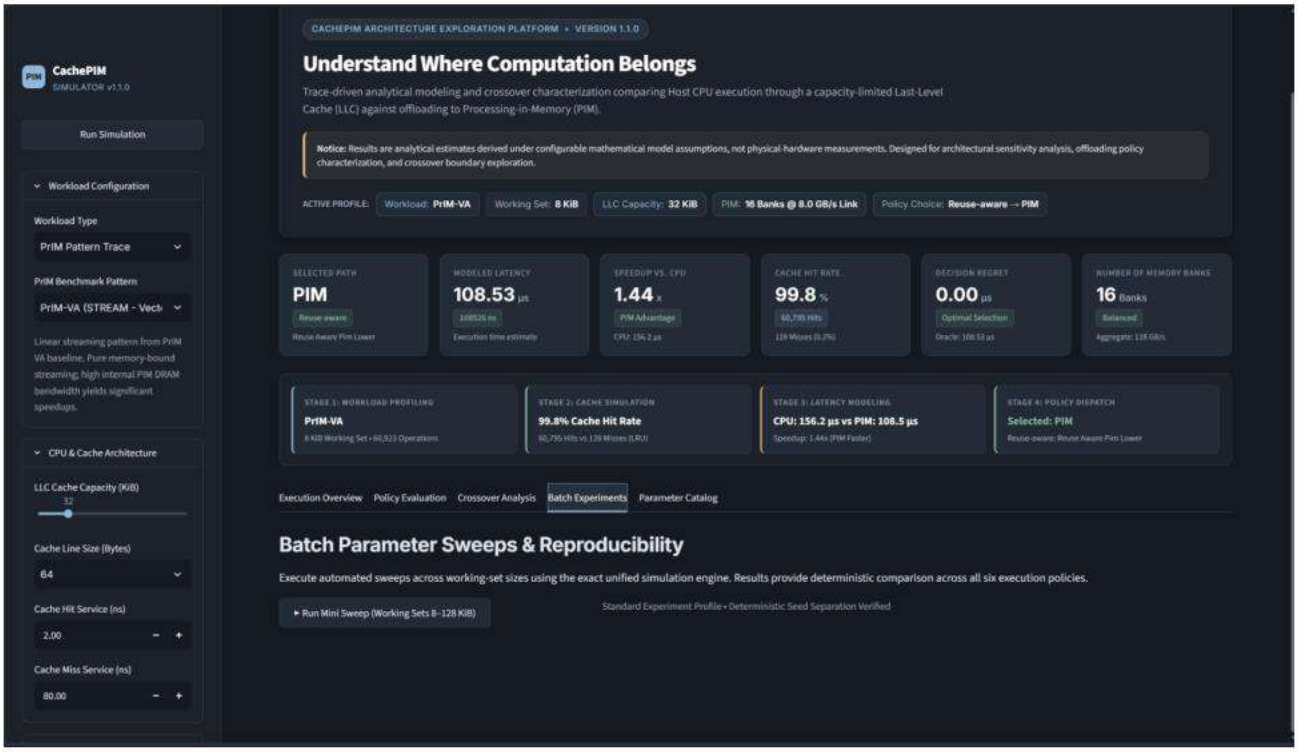


FIGURE 12. **Frontend screenshot 10: Computation placement overview and summary metrics.**

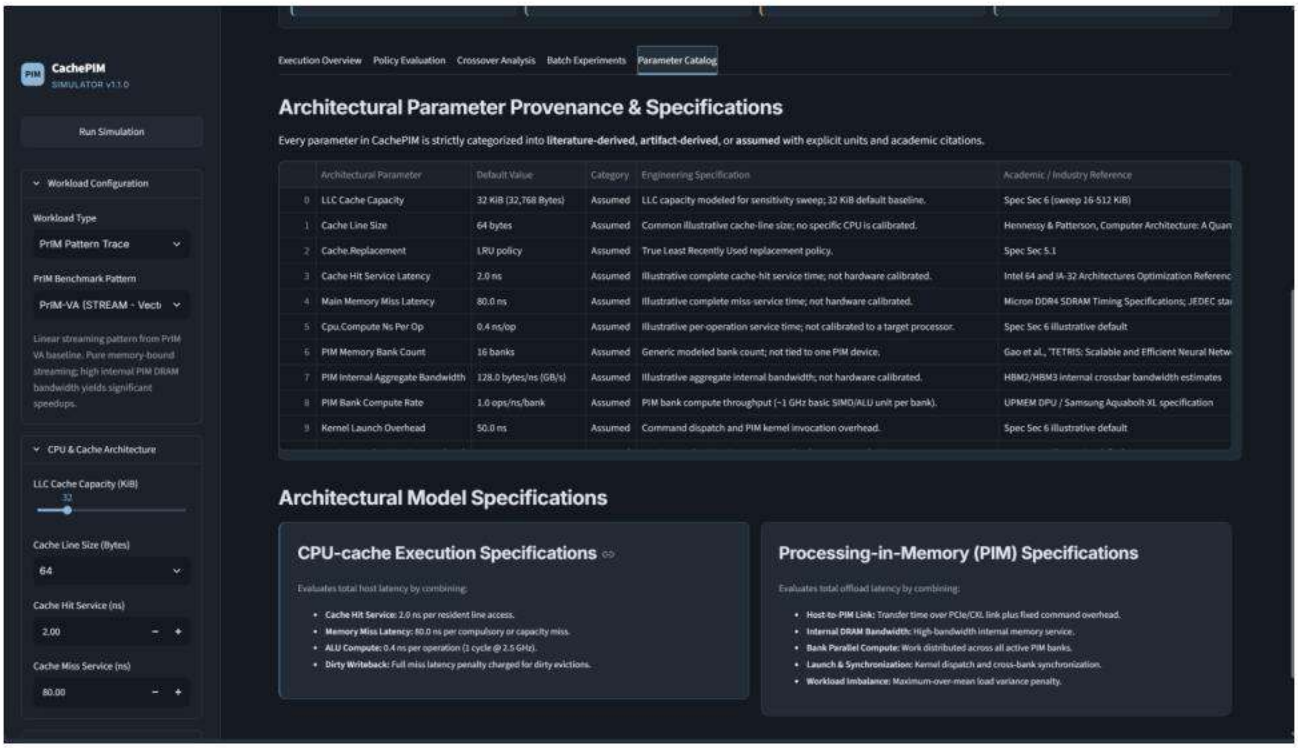


FIGURE 13. Frontend screenshot 11: Architectural parameter provenance and specifications.

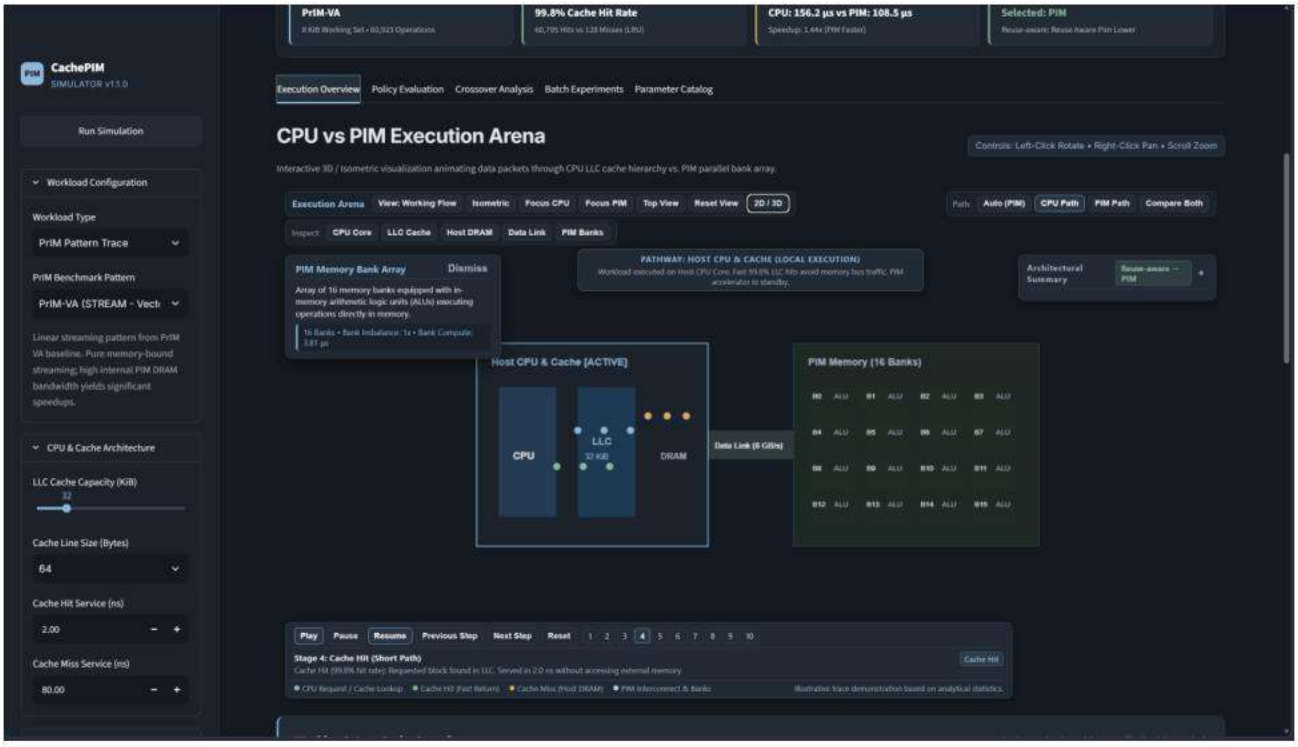


FIGURE 14. Frontend screenshot 12: Execution arena: alternate transfer configuration.



FIGURE 15. **Frontend screenshot 13: Execution arena: CPU/PIM state comparison.**

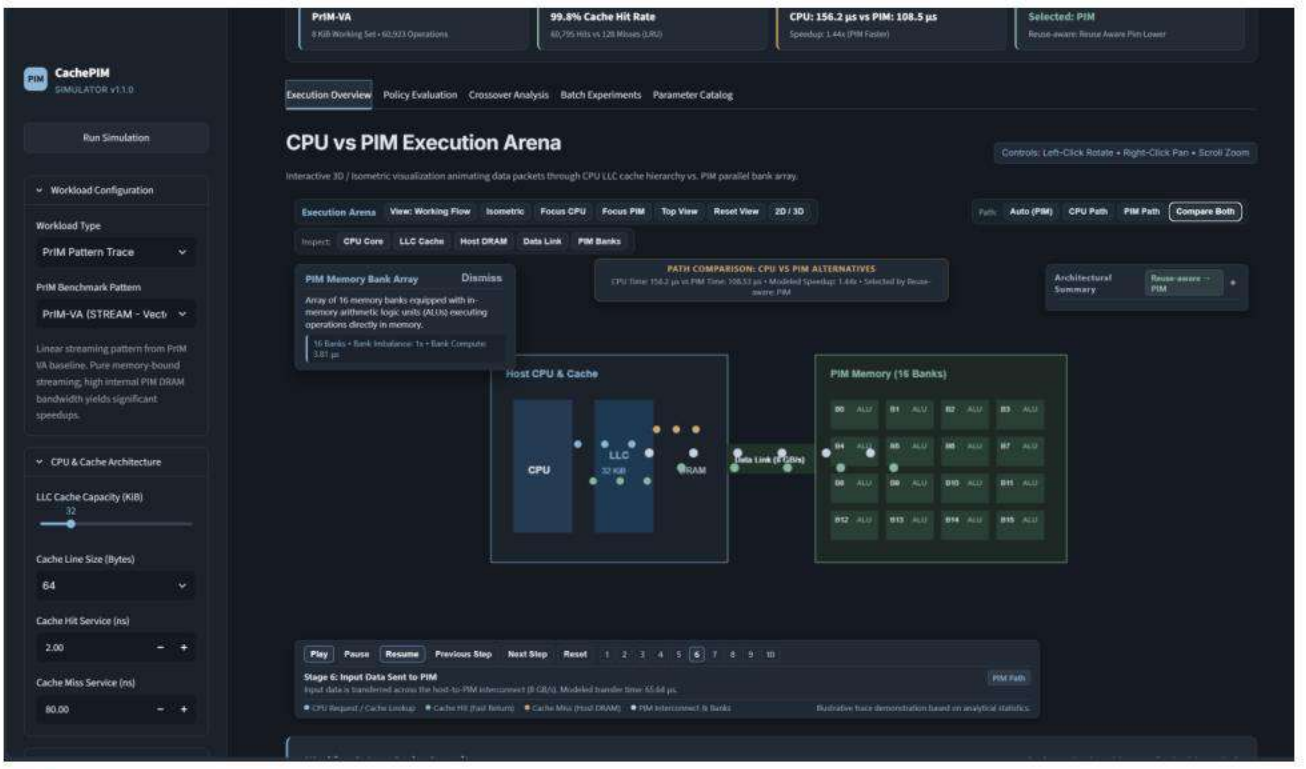


FIGURE 16. **Frontend screenshot 14: Execution arena: bank activity configuration.**

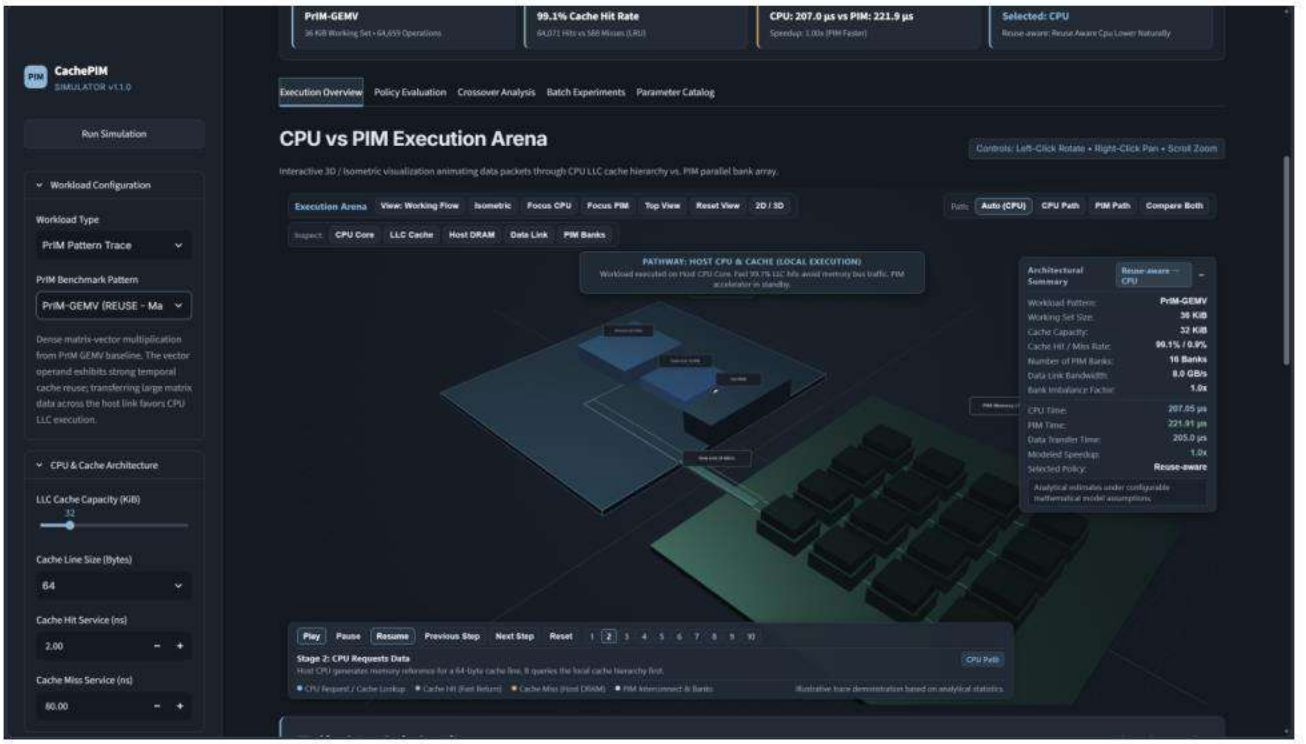


FIGURE 17. Frontend screenshot 15: Execution arena: 3D architectural overview.

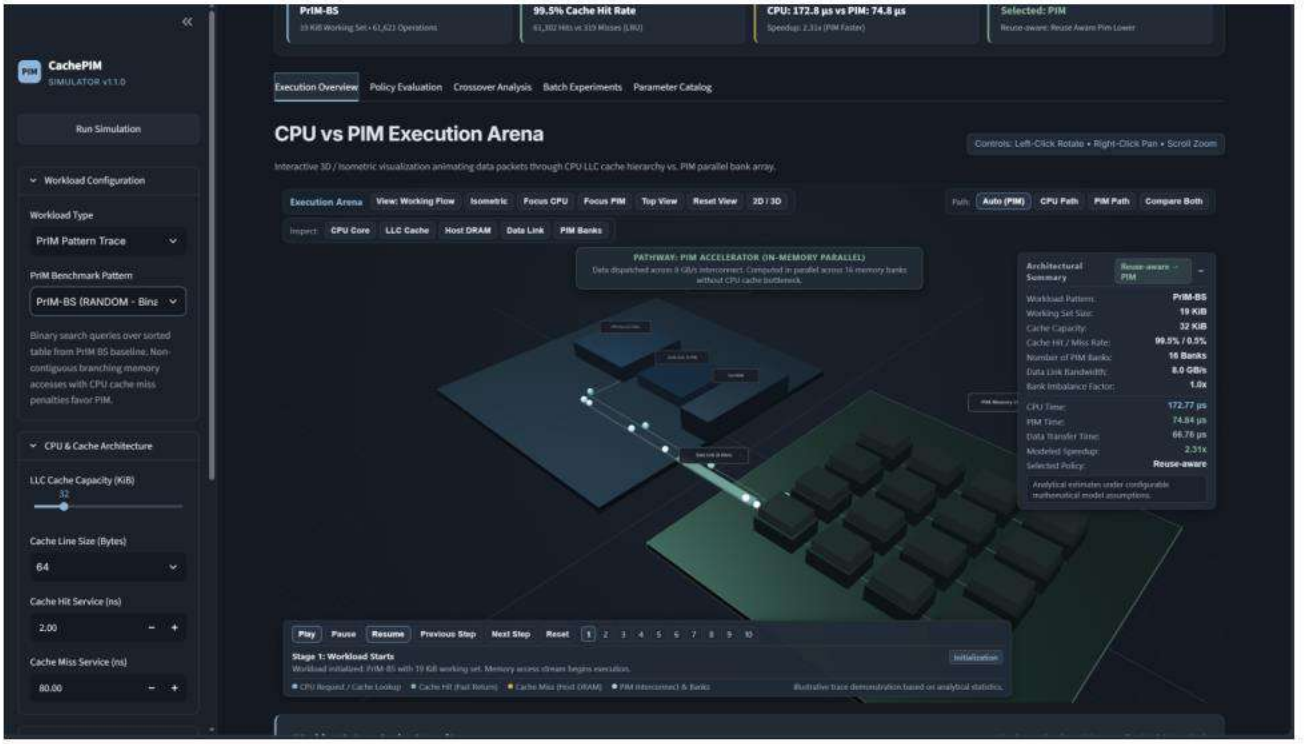


FIGURE 18. Frontend screenshot 16: Execution arena: model state and controls.



FIGURE 19. **Frontend screenshot 17: Execution arena: alternate 3D view.**



FIGURE 20. **Frontend screenshot 18: Component-level cost breakdown.**

- **Harsha Reddy:** Interactive 3D WebGL volumetric execution arena, Streamlit dashboard interface, and UI component architecture.
- **Bhuvaneswari M:** Research supervision, methodology validation, architectural modeling review, and manuscript finalization.

ACKNOWLEDGMENT

The authors thank the School of Computer Science and Engineering (SCOPE) at Vellore Institute of Technology for providing computational facilities.

REFERENCES

[1] K. Asifuzzaman, N. R. Miniskar, A. R. Young, F. Liu, and J. S. Vetter, "A survey on processing-in-memory techniques: Advances and challenges," *Memories - Materials, Devices, Circuits and Systems*, vol. 4, Art. no. 100022, Jul. 2023, doi: 10.1016/j.memori.2022.100022.

[2] Y.-K. Kim, B. Moon, D.-S. Kim, and Y.-J. Jang, "Cache memory organization for processing in memory," *IEICE Electronics Express*, vol. 16, no. 16, Art. no. 20190393, 2019, doi: 10.1587/ELEX.16.20190393.

[3] A. Devic, S. B. Rai, A. Sivasubramaniam, A. Akel, S. Eilert, and J. Eno, "To PIM or not for emerging general purpose processing in DDR memory systems," in *Proc. 49th Annu. Int. Symp. Computer Architecture (ISCA)*, 2022, pp. 231–244, doi: 10.1145/3470496.3527431.

[4] N. Hajinazar, G. F. Oliveira, S. Gregorio, J. D. Ferreira, N. M. Ghiasi, M. Patel, M. Alser, S. Ghose, J. Gómez-Luna, and O. Mutlu, "SIMDRAM: An end-to-end framework for bit-serial SIMD computing in DRAM," in *Proc. 26th ACM Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2021, pp. 329–345, doi: 10.1145/3445814.3446749.

[5] X. Peng, Y. Wang, and M.-C. Yang, "CHOPPER: A compiler infrastructure for programmable bit-serial SIMD processing using memory in DRAM," in *Proc. IEEE Int. Symp. High-Performance Computer Architecture (HPCA)*, 2023, pp. 1275–1288, doi: 10.1109/HPCA56546.2023.10071070.

[6] X. Wang, X. Sun, Y. Han, and X. Chen, "PIMSIM-NN: An ISA-based simulation framework for processing-in-memory accelerators," in *Proc. Design, Automation & Test in Europe Conf. & Exhibition (DATE)*, 2024, pp. 1–6, doi: 10.23919/DATE58400.2024.10546788.

[7] F. A. Siddique et al., "Architectural modeling and benchmarking for digital DRAM PIM," in *Proc. IEEE Int. Symp. Workload Characterization (IISWC)*, 2024, pp. 247–261, doi: 10.1109/IISWC63097.2024.00030.

[8] X. Sun, X. Wang, W. Li, Y. Han, and X. Chen, "PIMCOMP: An end-to-end DNN compiler for processing-in-memory accelerators," *IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems*, vol. 44, no. 5, pp. 1745–1759, 2025, doi: 10.1109/TCAD.2024.3496847.

[9] D. Guo, M. Gholamrezaei, M. Hofmann, A. Venkat, Z. Zhang, and K. Skadron, "PIMsynth: A unified compiler framework for bit-serial processing-in-memory architectures," *IEEE Computer Architecture Letters*, vol. 24, no. 2, pp. 277–280, 2025, doi: 10.1109/LCA.2025.3600588.

[10] H. Geng, T. S. Pereira, X. Lu, X.-H. Sun, M. Niemier, and X. S. Hu, "VIPER: Architecture-aware performance modeling for processing-in-memory design-space exploration," in *Proc. IEEE/ACM Int. Conf. Computer-Aided Design (ICCAD)*, 2026, pp. 1–9.

[11] J. Zhang, H. Wang, and E. Sadredini, "Meta-PIM: An end-to-end compiler for configurable processing-in-memory," *IEEE Computer Architecture Letters*, vol. 25, no. 2, pp. 112–116, Jul.-Dec. 2026, doi: 10.1109/LCA.2026.3709790.

[12] P. Yang, S. Durvasula, I. Fernandez, M. Sadrosadati, O. Mutlu, G. Pekhimenko, and C. Giannoula, "DCC: Data-centric compilation of machine learning kernels for processing-in-memory architectures," in *Proc. 53rd ACM/IEEE Annu. Int. Symp. Computer Architecture (ISCA)*, 2026, pp. 2582–2599, doi: 10.1109/ISCA66397.2026.00180.

[13] A. S. Rana, V. Mor, A. Raut, H. Nethra, and H. Reddy, "CachePIM-Analyser: Open-Source CPU-PIM Architectural Simulation Suite," 2026. [Online]. Available: `https://github.com/AdityaSRana/CachePIM-Analyse`

[14] J. Gómez-Luna, I. El Hajj, I. Fernandez, C. Giannoula, G. F. Oliveira, and O. Mutlu, "Benchmarking a new paradigm: Experimental analysis and characterization of a real processing-in-memory system," *IEEE Access*, vol. 10, pp. 52565–52608, 2022, doi: 10.1109/ACCESS.2022.3174101.

[15] O. Mutlu, S. Ghose, J. Gómez-Luna, and R. Ausavarungnirun, "A modern primer on processing in memory," in *Emerging Computing: From Devices to Systems*, Springer, 2023, pp. 171–243.

[16] C. Giannoula, I. Fernandez, R. Quislant, E. Gutiérrez, O. Plata, and O. Mutlu, "SynCron: Efficient synchronization support for near-data-processing architectures," in *Proc. IEEE Int. Symp. High-Performance Computer Architecture (HPCA)*, 2021, pp. 263–276, doi: 10.1109/HPCA51647.2021.00031.

[17] A. Boroumand et al., "CoNDA: Efficient cache coherence support for near-data computing," in *Proc. 46th Annu. Int. Symp. Computer Architecture (ISCA)*, 2019, pp. 629–642, doi: 10.1145/3307650.3322259.

[18] S. Ghose et al., "Processing-in-memory: A workload-driven perspective," *IBM Journal of Research and Development*, vol. 63, no. 6, pp. 3:1–3:19, 2019, doi: 10.1147/JRD.2019.2934048.

[19] K. Hsieh, E. Ebrahimi, G. Kim, N. Chatterjee, M. O'Connor, N. Vijaykumar, O. Mutlu, and S. W. Keckler, "Transparent offloading and mapping (TOM): Enabling programmer-transparent near-data processing in GPU systems," in *Proc. 43rd Annu. Int. Symp. Computer Architecture (ISCA)*, 2016, pp. 204–216, doi: 10.1145/2925426.2926288.

[20] G. F. Oliveira et al., "DAMOV: A new methodology and benchmark suite for evaluating data movement bottlenecks," *IEEE Access*, vol. 9, pp. 134443–134473, 2021, doi: 10.1109/ACCESS.2021.3115456.

[21] L. Ke et al., "RecNMP: Accelerating personalized recommendation with near-memory processing," in *Proc. 47th Annu. Int. Symp. Computer Architecture (ISCA)*, 2020, pp. 790–803, doi: 10.1109/ISCA45697.2020.00071.
[22] Y. Kwon et al., "25.4 A 20nm 6GB function-in-memory DRAM, with a floating-point programmable accelerator for computing-in-memory applications," in *IEEE Int. Solid-State Circuits Conf. (ISSCC)*, 2021, pp. 350–352, doi: 10.1109/ISSCC42613.2021.9365780.
[23] J. Lee et al., "Hardware architecture and software stack for PIM-based neural network accelerators," *IEEE Micro*, vol. 41, no. 6, pp. 49–57, 2021, doi: 10.1109/MM.2021.3111425.
[24] S. Lee et al., "AiM: A 16Gb 1.0 GHz GDDR6 accelerator-in-memory for energy-efficient deep learning," in *IEEE Int. Solid-State Circuits Conf. (ISSCC)*, 2022, pp. 1–3, doi: 10.1109/ISSCC42614.2022.9731671.
[25] T. Grosser, A. Groesslinger, and C. Lengauer, "Polly—Performing polyhedral loop transformations on a high-level IL," in *Proc. First Int. Workshop on Polyhedral Compilation Techniques (IMPACT)*, 2011, pp. 1–8.
[26] C. Lattner, M. Amini, U. Bondhugula, A. Cohen, A. Davis, J. Pienaar, R. Riddle, T. Shpeisman, N. Vasilache, and O. Zinenko, "MLIR: Scaling compiler infrastructure for domain-specific computation," in *Proc. IEEE/ACM Int. Symp. Code Generation and Optimization (CGO)*, 2021, pp. 2–14, doi: 10.1109/CGO51591.2021.9370308.
[27] N. Nethercote and J. Seward, "Valgrind: A framework for heavyweight dynamic binary instrumentation," in *Proc. ACM SIGPLAN Conf. Programming Language Design and Implementation (PLDI)*, 2007, pp. 89–100, doi: 10.1145/1250734.1250746.
[28] G. Pekhimenko, V. Seshadri, Y. Kim, H. Xin, O. Mutlu, P. B. Gibbons, M. A. Kozuch, and T. C. Mowry, "Linearly compressed pages: A main memory compression framework," in *Proc. 46th Annu. IEEE/ACM Int. Symp. Microarchitecture (MICRO)*, 2013, pp. 172–184, doi: 10.1145/2540708.2540724.
[29] M. Gao, G. Ayers, and C. Kozyrakis, "Practical near-data processing for in-memory analytics," in *Proc. 20th Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2015, pp. 113–126, doi: 10.1145/2694344.2694386.
[30] J. Ahn, S. Hong, S. Yoo, O. Mutlu, and K. Choi, "A scalable processing-in-memory accelerator for parallel graph processing," in *Proc. 42nd Annu. Int. Symp. Computer Architecture (ISCA)*, 2015, pp. 105–117, doi: 10.1145/2749469.2750386.